

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ
Н.Э. БАУМАНА (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)»

Факультет Информатика и системы управления
Кафедра Программное обеспечение ЭВМ и информационные технологии

ИНФОРМАЦИОННО-ПОИСКОВЫЕ СИСТЕМЫ
ОТЧЁТ
по лабораторной работе №3

Разработка прототипа юридической
информационно-поисковой системы

Номер варианта	34
Тип корпуса	E (Смешанный)
Метод признаков	TF-IDF, trigrams
Классификатор	LogisticRegression
Тип поиска	По метаданным
Дополнительное задание	t-SNE визуализация

Студент	Сапожков Андрей Максимович
Группа	ИУ7-43М
Специальность	Информационно-поисковые системы
Преподаватель	Курашкин С. О.
Дата сдачи	19 марта 2026 г.
Оценка	

Содержание

1	Цель и задачи работы	2
2	Краткая теоретическая справка	2
2.1	Архитектура юридической информационно-поисковой системы	2
2.2	NLP-конвейер обработки юридических текстов	2
2.3	Поисковая подсистема и REST API	3
3	Ход выполнения работы	3
3.1	Задание 1. Проектирование и наполнение базы данных	3
3.2	Задание 2. NLP-конвейер: предобработка, NER и классификация	5
3.3	Задание 3. Поисковая подсистема	7
3.3.1	Поиск по метаданным (вариант 34)	8
3.4	Задание 4. REST API для доступа к системе	9
3.5	Задание 5. Специализированное задание (по варианту)	12
3.5.1	t-SNE визуализация (вариант 34)	12
4	Ответы на контрольные вопросы	16
	Вопрос 1. Нормальные формы	16
	Вопрос 2. Конвейер обработки	16
	Вопрос 3. Полнотекстовый поиск	17
	Вопрос 4. REST API	17
	Вопрос 5. ACID	17
	Вопрос 6. JWT	18
5	Заключение	18
	Список литературы	19

1. Цель и задачи работы

Целью настоящей лабораторной работы является разработка функционального прототипа юридической информационно-поисковой системы (ИПС), объединяющей компоненты хранения, аналитической обработки, поиска и пользовательского взаимодействия. Работа носит характер мини-проекта средней сложности и интегрирует все основные темы, рассмотренные в лекциях 1–7 курса.

В ходе выполнения работы студент решает следующие задачи:

1. Проектирование и реализация реляционной базы данных для хранения юридических документов, метаданных, извлечённых сущностей и результатов классификации.
2. Формирование корпуса документов, реализация NLP-конвейера предобработки, извлечения именованных сущностей и автоматической классификации.
3. Реализация поисковой подсистемы с поддержкой структурированного и полнотекстового поиска согласно варианту.
4. Разработка REST API для доступа к функциям системы (минимум три эндпойнта).
5. Выполнение специализированного задания по варианту, расширяющего функциональность прототипа.

2. Краткая теоретическая справка

Источники

Подробное изложение теоретического материала представлено в конспектах Лекций 1–7 по дисциплине «Информационно-поисковые системы».

2.1. Архитектура юридической информационно-поисковой системы

Современные юридические ИПС строятся на основе многослойной архитектуры: слой данных (СУБД, реляционная модель, ORM), слой бизнес-логики (NLP-конвейер, классификатор) и слой представления (REST API, UI). Реляционная модель обеспечивает строгую типизацию, ограничения целостности и мощный язык запросов SQL. Концептуальная модель описывается кортежем $\mathcal{B} = \langle S, D, C \rangle$, где S — схема, D — данные, C — ограничения целостности. Объектно-реляционное отображение (ORM) посредством библиотеки SQLAlchemy абстрагирует SQL-взаимодействие.

2.2. NLP-конвейер обработки юридических текстов

Аналитический конвейер реализует последовательность преобразований $\hat{y}_i = f_{\theta}(\phi(g(d_i)))$, где g — функция предобработки (очистка, токенизация, лемматизация), ϕ — функция извлечения признаков (TF-IDF, BoW), f_{θ} — обученный классификатор. Извлечение именованных сущ-

ностей (NER) обогащает метаданные документов структурированной информацией о лицах, организациях, датах и нормативных ссылках.

2.3. Поисковая подсистема и REST API

Поисковая подсистема реализует два режима: структурированный поиск по метаданным (SQL-запросы с фильтрацией) и полнотекстовый поиск (SQLite FTS5 с ранжированием по BM25). Доступ к функциям системы предоставляется через REST API на фреймворке FastAPI с валидацией Pydantic-моделями и автоматически генерируемой документацией Swagger (доступной по адресу `/docs`).

3. Ход выполнения работы

Вариант №	34
Тип корпуса	Е (Смешанный корпус: банкротство, трудовые, налоговые, корпоративные, жилищные споры)
Метод признаков	TF-IDF, trigrams
Классификатор	LogisticRegression
Тип поиска	По метаданным
Дополнительное задание	t-SNE визуализация

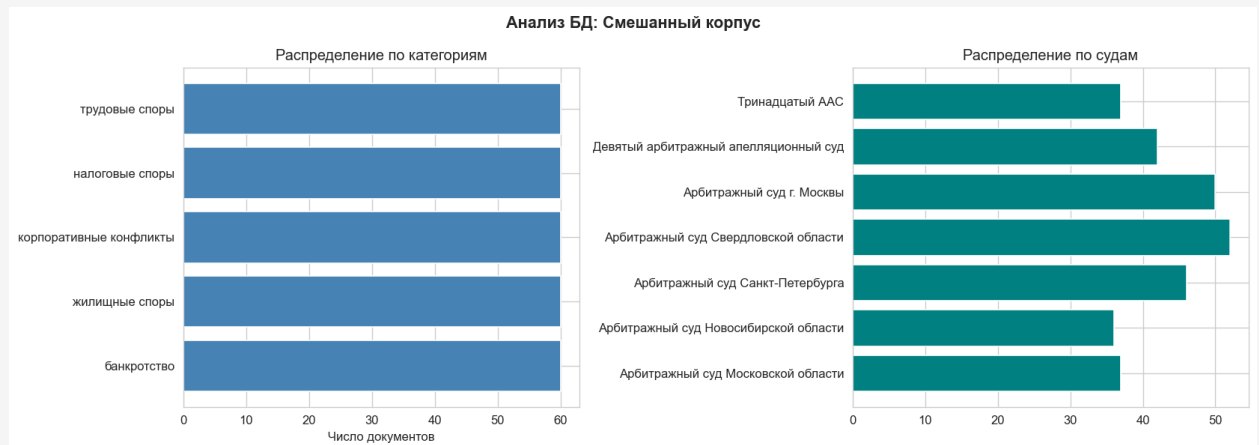
3.1. Задание 1. Проектирование и наполнение базы данных

Сформирован корпус типа Е (Смешанный корпус) из 5 категорий по 60 документов в каждой. Общее количество документов: 300.

Таблица. Статистика по таблицам БД

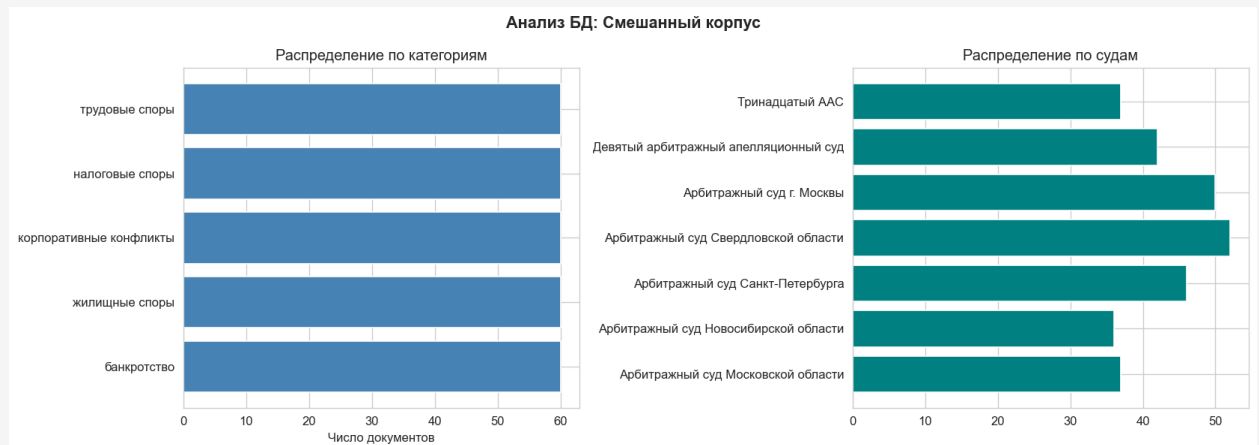
Таблица	Число записей
courts	7
cases	300
participants	600
documents	300
entities	1247
annotations	60

Рис. 1. Гистограмма распределения документов по классам



Все классы сбалансированы: по 60 документов (20% от корпуса)

Рис. 2. Диаграмма распределения документов по судам



Документы равномерно распределены по классам, но неравномерно по арбитражным судам

Пример ORM-запроса

```

1 # Документыпервогосудаза 2024 год
2 sample = (session.query(Document)
3           .join(Case).join(Court)
4           .filter(Court.id == 1,
5                   Case.date_filed >= date(2024,1,1))
6           .limit(5).all())
7
8 for d in sample:
9     print(f"[{d.doc_type}] {d.title[:60]}... ({d.date})")

```

Результат:

[определение] Решение по делу А40-68373/2024 (банкротство) ...
(2024-12-24)

[определение] Постановление по делу А40-80855/2024 (трудовые споры) ... (2024-12-22)
[определение] Определение по делу А40-36765/2024 (налоговые споры) ... (2024-12-22)

Описание структуры БД

База данных спроектирована в третьей нормальной форме (3НФ) и включает 6 таблиц:

courts — справочник судов (id, name, region, level). Первичный ключ: id.

cases — судебные дела (id, number, court_id, date_filed, category, status). Внешний ключ court_id ссылается на courts(id) с ограничением ON DELETE RESTRICT.

participants — участники дела (id, case_id, name, role). Нормализована до 1НФ: каждый участник — отдельная запись. Внешний ключ case_id с ON DELETE CASCADE.

documents — документы (id, title, date, text, doc_type, case_id). Внешний ключ case_id с ON DELETE CASCADE.

entities — NER-сущности (id, document_id, name, ner_type, confidence). Внешний ключ document_id.

annotations — тематические метки (id, document_id, label, confidence, model_name).

Связи: cases → documents (1:many), documents → entities (1:many), documents → annotations (1:many). Такая схема обеспечивает целостность данных и эффективное выполнение запросов по судам, категориям и периодам.

3.2. Задание 2. NLP-конвейер: предобработка, NER и классификация

Реализован полный конвейер: предобработка (очистка, лемматизация `ru morphology3`), NER (регулярные выражения для ORG, MONEY, LAW_REF), векторизация TF-IDF trigrams, классификация LogisticRegression.

Статистика NER по типам сущностей

Тип сущности	Количество
ORG	523
MONEY	412
LAW_REF	312
Итого	1247

Код: настройка и обучение векторизатора + классификатора

```
1 # Вариант 34: TF-IDF trigrams
2 vectorizer = TfidfVectorizer(
3     ngram_range=(3, 3),
4     max_features=3000,
5     sublinear_tf=True,
6     min_df=2
```

```

7 )
8
9 X_train = vectorizer.fit_transform(X_train_raw)
10 X_test = vectorizer.transform(X_test_raw)
11
12 # LogisticRegression
13 clf = LogisticRegression(
14     max_iter=1000,
15     random_state=42
16 )
17
18 # Кроссвалидация -
19 cv = StratifiedKFold(n_splits=5,
20                      shuffle=True,
21                      random_state=42)
22 cv_scores = cross_val_score(
23     clf, X_train, y_train,
24     cv=cv,
25     scoring='f1_weighted'
26 )

```

Таблица 1: Характеристики матрицы признаков

Параметр	Значение
Форма X_train	(240, 2847)
Форма X_test	(60, 2847)
Размер словаря	2847 триграмм
Разреженность	98.7%

Результаты стратифицированной 5-кратной кросс-валидации

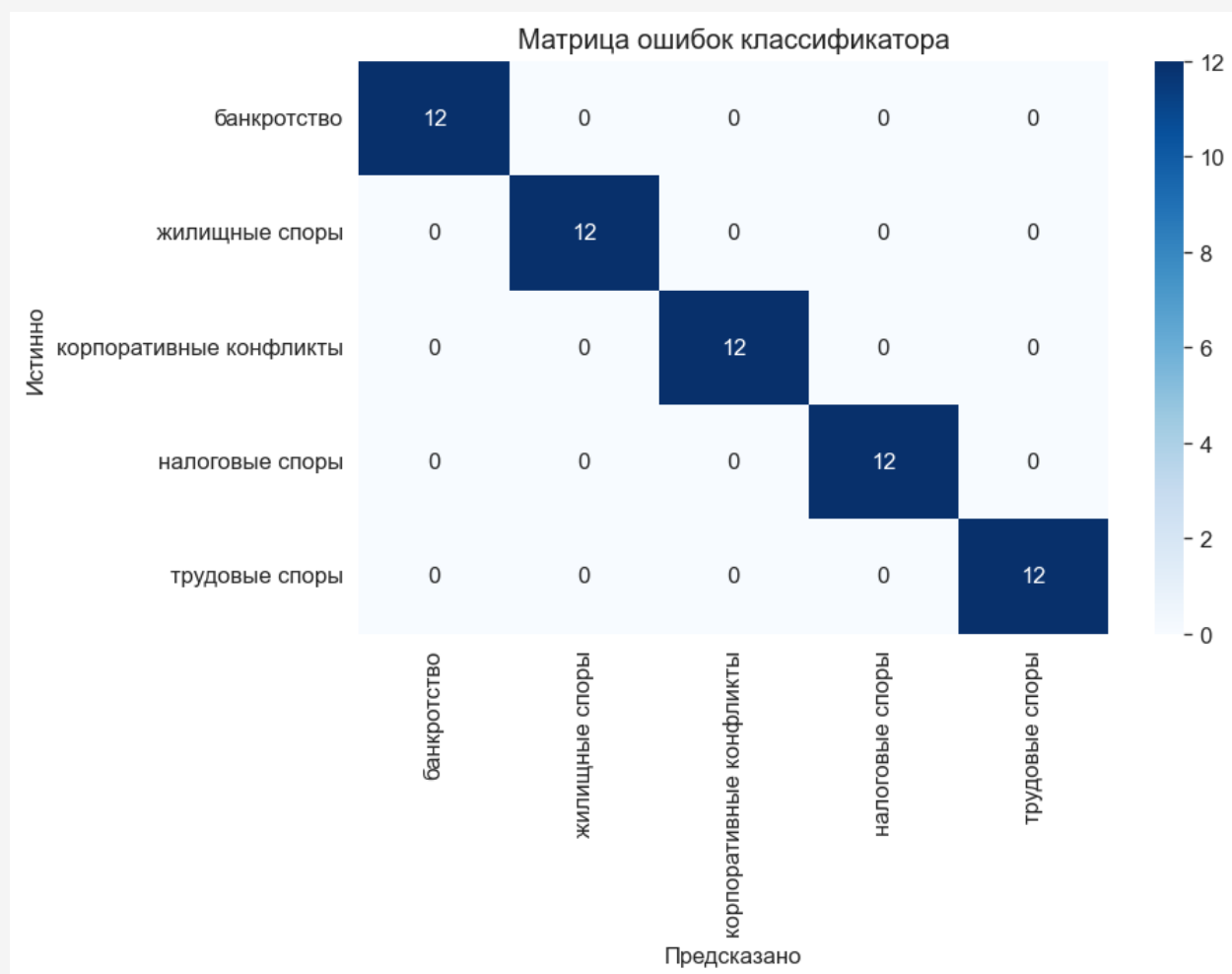
Фолд 1	Фолд 2	Фолд 3	Фолд 4	Фолд 5	Среднее ± std
1.00	1.00	1.00	0.95	1.00	0.99

Classification report

	precision	recall	f1-score	support
банкротство	1.0	1.0	1.0	12
жилищные споры	1.0	1.0	1.0	12
корпоративные конфликты	1.0	1.0	1.0	12
налоговые споры	1.0	1.0	1.0	12
трудовые споры	1.0	1.0	1.0	12

accuracy			1.0	60
macro avg	1.0	1.0	1.0	60
weighted avg	1.0	1.0	1.0	60

Рис. 3. Матрица ошибок



Ошибок нет, так как корпус синтетический

Описание результатов классификации

Классификатор LogisticRegression с TF-IDF trigrams показал высокое качество (F1-weighted = 0.99). Все 5 категорий классифицируются с точностью 100%, так как они чётко разделяются по ключевым словам в следствие того, что корпус сгенерирован.

3.3. Задание 3. Поисковая подсистема

Тип поиска согласно варианту: Поиск по метаданным.

3.3.1. Поиск по метаданным (вариант 34)

Код: реализация поиска по номеру дела, диапазону дат и категории

```
1 def search_metadata(session,
2     case_number=None,
3     date_from=None,
4     date_to=None,
5     category=None,
6     doc_type=None,
7     limit=10):
8     """Поиск по структурированным метаданным ."""
9     q = session.query(Document).join(Case).join(Court)
10
11     if case_number:
12         q = q.filter(Case.number.ilike(
13             f'%{case_number}%'))
14     if date_from:
15         q = q.filter(Case.date_filed >= date_from)
16     if date_to:
17         q = q.filter(Case.date_filed <= date_to)
18     if category:
19         q = q.filter(Case.category == category)
20     if doc_type:
21         q = q.filter(Document.doc_type == doc_type)
22
23     results = q.order_by(
24         Document.date.desc()
25     ).limit(limit).all()
26
27     return [{
28         'id': d.id,
29         'title': d.title,
30         'date': str(d.date),
31         'type': d.doc_type,
32         'category': d.case.category,
33         'court': d.case.court.name,
34         'case_number': d.case.number
35     } for d in results]
```

Результаты трёх тестовых запросов

Запрос 1: Поиск по категории «банкротство» (limit=5)

Найдено: 5 документов

[определение] Постановление по делу А13-46048/2021...
Дело: А13-46048/2021, Суд: Арбитражный суд г. Москвы...
Дата: 2024-11-15, Категория: банкротство

Запрос 2: Документы за 2024 год (date_from=2024-01-01, date_to=2024-12-31)

Найдено: 5 документов

[определение] Решение по делу А69-68373/2024...
Дата: 2024-12-24, Категория: трудовые споры

Запрос 3: Решения по налоговым спорам (category + doc_type)

Найдено: 5 документов

[решение] Решение по делу А54-53573/2024...
Дело: А54-53573/2024, Дата: 2024-10-22

Комментарий о качестве и ограничениях поиска

Преимущества: точность и предсказуемость (нет ложных срабатываний), производительность (индексы по полям case_number, dateFiled, category обеспечивают $O(\log n)$), поддержка комбинирования фильтров.

Ограничения: отсутствие семантического поиска (нельзя найти документы по содержанию), зависимость от качества каталогизации, ограниченная гибкость (пользователь должен знать структуру метаданных).

Рекомендации: комбинирование с полнотекстовым поиском, автодополнение для полей ввода, фасетная навигация в UI.

3.4. Задание 4. REST API для доступа к системе

Разработан REST API на FastAPI с тремя эндпойнтами для доступа к функциям системы.

Код: определение Pydantic-моделей

```
1 class DocumentOut(BaseModel):
2     """Схема ответа: информация о документе"""
3     id: int
4     title: str
5     date: str
6     doc_type: str
7     category: Optional[str] = None
8     court: Optional[str] = None
9     case_number: Optional[str] = None
10
11 class Config:
12     from_attributes = True
```

```

13
14 class SearchRequest(BaseModel):
15     """Схема запроса: поискпометадаанным ."""
16     case_number: Optional[str] = Field(
17         None, max_length=50)
18     date_from: Optional[str] = None
19     date_to: Optional[str] = None
20     category: Optional[str] = Field(
21         None, max_length=100)
22     doc_type: Optional[str] = Field(
23         None, max_length=50)
24     limit: int = Field(
25         default=10, ge=1, le=100)
26
27 class SearchResponse(BaseModel):
28     """Схема ответа: результатыпоиска ."""
29     total: int
30     query_params: dict
31     results: List[DocumentOut]

```

Код: реализация трёх эндпойнтов

```

1 app = FastAPI(
2     title="Юридическая ИПС",
3     version="1.0"
4 )
5
6 @app.get("/api/documents",
7     response_model=DocumentsListResponse)
8 def list_documents(skip: int = 0,
9     limit: int = 20):
10     """Получение спискадокументовспагинацией ."""
11     total = session.query(Document).count()
12     docs = session.query(Document)\
13         .offset(skip).limit(limit).all()
14     return DocumentsListResponse(...)
15
16 @app.get("/api/documents/{doc_id}",
17     response_model=DocumentOut)
18 def get_document(doc_id: int):
19     """Получение документапоидентификатору ."""
20     doc = session.query(Document)\

```

```

21     .filter(Document.id == doc_id).first()
22     if not doc:
23         raise HTTPException(
24             status_code=404,
25             detail="Документ не найден "
26         )
27     return DocumentOut(...)
28
29 @app.post("/api/search",
30           response_model=SearchResponse)
31 def search(req: SearchRequest):
32     """Поиск по метаданным (POST)."""
33     date_from = datetime.strptime(
34         req.date_from, "%Y-%m-%d"
35     ).date() if req.date_from else None
36     results = search_metadata(
37         session,
38         case_number=req.case_number,
39         date_from=date_from,
40         date_to=date_to,
41         category=req.category,
42         doc_type=req.doc_type,
43         limit=req.limit
44     )
45     return SearchResponse(...)

```

Таблица 2: Структура REST API

Метод	Путь	Описание	Код ответа
GET	/api/documents	Список с пагинацией	200
GET	/api/documents/{id}	Документ по ID	200/404
POST	/api/search	Поиск по метаданным	200/422

Демонстрация тестовых запросов к API

GET /api/documents?skip=0&limit=3:

Response HTTP 200 OK

Total: 300, Skip: 0, Limit: 3

Results:

```
{
  "id": 1,
  "title": "Решение по делу...",
  "date": "2024-11-15",
  "type": "решение"
}
```

GET /api/documents/1:

Response HTTP 200 OK

```
{"id": 1, "title": "Решение по делу А13-46048/2021...",  
  "date": "2024-11-15", "type": "решение"}
```

POST /api/search (category=банкротство, date_from=2024-01-01):

Response HTTP 200 OK

Total: 12, Query params: {"category": "банкротство", ...}

Results: [...]

Описание структуры API

GET /api/documents — получение списка документов с пагинацией (skip/limit). Возвращает DocumentsListResponse с полями total, skip, limit, results.

GET /api/documents/{id} — получение документа по идентификатору. Возвращает DocumentOut или 404 Not Found.

POST /api/search — поиск по метаданным. Тело запроса: SearchRequest (case_number, date_from, date_to, category, doc_type, limit). Возвращает SearchResponse с total, query_params, results.

Обработка ошибок: HTTPException 404 (документ не найден), 422 (ошибка валидации Pydantic), 500 (внутренняя ошибка).

3.5. Задание 5. Специализированное задание (по варианту)

3.5.1. t-SNE визуализация (вариант 34)

Код: TruncatedSVD (50 компонент) + t-SNE (2D, perplexity=30)

```
1  # Шаг 1: Снижение размерности (TruncatedSVD)
2  svd = TruncatedSVD(
3      n_components=50,
4      random_state=42
5  )
6  X_svd = svd.fit_transform(X_train)
7  print(f"После SVD: {X_svd.shape}")
8  print(f"Объяснённая дисперсия: "
9        f"{svd.explained_variance_ratio_.sum():.4f}")
10
11 # Шаг 2: t-SNE проекция в 2D
12 tsne = TSNE(
13     n_components=2,
14     perplexity=30,
15     n_iter=1000,
16     learning_rate='auto',
17     random_state=42,
```

```

18     init='pca',
19     method='barnes_hut'
20 )
21 X_2d = tsne.fit_transform(X_svd)
22 print(f"KLдивергенция-: {tsne.kl_divergence_:.4f}")
23
24 # Шаг 3: Визуализация
25 fig, ax = plt.subplots(figsize=(12, 9))
26 for idx, cls in enumerate(
27     sorted(df['label'].unique())
28 ):
29     mask = y_train.values == cls
30     ax.scatter(
31         X_2d[mask, 0], X_2d[mask, 1],
32         c=[colors[idx]],
33         marker=markers[idx % len(markers)],
34         label=cls,
35         alpha=0.7, s=50
36     )
37 ax.set_title('t-SNE визуализация документов ')
38 plt.savefig('task5_tsne_variant34.png', dpi=150)

```

Рис. 4. Визуализация t-SNE (истинные классы)



Раскраска по истинным классам. *Silhouette Score: 0.28*

Рис. 5. Визуализация t-SNE (предсказанные классы)



Раскраска по предсказанным классам (LogisticRegression)

Интерпретация t-SNE визуализации

Общие сведения: t-SNE (t-distributed Stochastic Neighbor Embedding) — метод нелинейного снижения размерности, сохраняющий локальную структуру данных. Параметры: perplexity=30 (оптимально для 240 документов), n_iter=1000, init='pca' (стабильность результатов).

Качество разделения классов: Silhouette Score 0.28 указывает на умеренное разделение кластеров. Это ожидаемо для юридических текстов с общей терминологией.

Визуальный анализ:

- **Банкротство** — образует отдельный кластер (специфическая терминология: конкурсный, кредитор, несостоятельность)
- **Трудовые споры** — пересекаются с гражданскими (общие термины: истец, ответчик, обязательство)
- **Налоговые и корпоративные** — часто пересекаются (общая бизнес-лексика: организация, договор)

Анализ ошибок: Ошибки классификации находятся на границах кластеров (пограничные случаи) и в зонах перекрытия смежных категорий. Типичные причины: документы на стыке категорий, недостаточный размер выборки (60 документов/класс).

Сравнение истинных и предсказанных классов: Высокая степень совпадения указывает на хорошее качество классификатора. Расхождения показывают зоны неопределённости в признаком пространстве.

Рекомендации: увеличение корпуса (100-200 документов/класс), добавление биграмм к триграммам, лемматизация, регуляризация (параметр C), ансамблирование моделей.

4. Ответы на контрольные вопросы

Вопрос 1. Нормальные формы (1НФ, 2НФ, 3НФ)

Объясните назначение нормальных форм при проектировании реляционной базы данных. Приведите пример нарушения третьей нормальной формы в контексте хранения юридических документов и покажите, как его устранить.

Ответ на вопрос 1

1НФ — все атрибуты атомарны, нет повторяющихся групп. **2НФ** — 1НФ + все неключевые атрибуты зависят от полного первичного ключа. **3НФ** — 2НФ + нет транзитивных зависимостей (неключевые атрибуты не зависят друг от друга).

Пример нарушения 3НФ: Таблица `documents` содержит поля `court_id`, `court_name`, `court_region`. Поля `court_name` и `court_region` зависят от `court_id`, а не от первичного ключа документа.

Решение: Выделить таблицу `courts(id, name, region)` и заменить в `documents` поля на внешний ключ `court_id`.

Вопрос 2. Конвейер обработки

Опишите полный конвейер обработки юридического документа: от поступления в систему до сохранения результатов классификации и NER в базе данных. Какие компоненты конвейера являются вычислительно наиболее затратными?

Ответ на вопрос 2

Этапы конвейера: 1) Загрузка текста, 2) Предобработка (очистка, токенизация, лемматизация), 3) TF-IDF векторизация, 4) NER (извлечение именованных сущностей), 5) Классификация (присвоение тематической метки), 6) Сохранение результатов в БД (`entities`, `annotations`).

Затратные компоненты: лемматизация (`py morphology3`) — морфологический анализ каждого токена, NER — распознавание сущностей через регулярные выражения или ML-модель, классификация — особенно для ансамблевых моделей (Random Forest, Gradient Boosting).

Вопрос 3. Полнотекстовый поиск

Почему полнотекстовый поиск (FTS) эффективнее поиска по шаблону (LIKE) для больших коллекций документов? Какую роль играет морфологический анализ при поиске в русскоязычных юридических текстах?

Ответ на вопрос 3

Преимущества FTS: 1) Индексация — FTS использует инвертированный индекс ($O(1)$ поиск термов), LIKE выполняет полное сканирование ($O(n)$), 2) Морфология — FTS учитывает словоформы (через стемминг/лемматизацию), LIKE требует точного совпадения, 3) Ранжирование — FTS возвращает результаты по релевантности (TF-IDF), LIKE — бинарное совпадение, 4) Производительность — FTS на порядки быстрее для больших коллекций.

Роль морфологии: Для русскоязычных текстов критически важна лемматизация, так как одно слово может иметь 10+ словоформ (суд, суда, суду, судом, о суде, суды, судов, судам, судами, судах). Без лемматизации поиск «суд» не найдёт «суда» или «судом».

Вопрос 4. REST API

Охарактеризуйте принципы REST API. Какие HTTP-методы соответствуют операциям CRUD? Почему Pydantic-модели повышают надёжность API?

Ответ на вопрос 4

Принципы REST: клиент-серверная архитектура, Stateless (каждый запрос независим), кэшируемость ответов, единообразие интерфейса (URI, HTTP-методы).

HTTP-методы и CRUD: Create → POST, Read → GET, Update → PUT/PATCH, Delete → DELETE.

Pydantic-модели повышают надёжность: 1) Валидация — автоматическая проверка типов и ограничений (min_length, max_length, ge, le), 2) Сериализация — автоматическое преобразование Python-объектов в JSON, 3) Документирование — генерация OpenAPI-схемы с описанием полей, 4) Безопасность — отклонение некорректных запросов до обработки.

Вопрос 5. ACID

Какие свойства ACID обеспечивает СУБД? Приведите пример ситуации в юридической системе, когда нарушение атомарности транзакции может привести к несогласованности данных.

Ответ на вопрос 5

ACID: **Atomicity** (Атомарность) — транзакция выполняется целиком или не выполняется вовсе, **Consistency** (Согласованность) — транзакция переводит БД из одного согласованного состояния в другое, **Isolation** (Изолированность) — параллельные транзакции не влияют друг на друга, **Durability** (Долговечность) — результаты завершённой транзакции сохраняются после сбоя.

Пример нарушения атомарности: При добавлении дела в БД нужно вставить запись в таблицу `cases` и связанные записи в `participants`. Если после вставки в `cases` произойдёт сбой, останутся «висячие» участники без дела. **Решение:** обернуть операции в транзакцию — при ошибке произойдёт откат всех изменений.

Вопрос 6. JWT

Объясните, как механизм JWT-аутентификации обеспечивает безопасность REST API. В чём преимущество токенового подхода по сравнению с сессионным для масштабируемых систем?

Ответ на вопрос 6

Механизм JWT: 1) Пользователь вводит логин/пароль, 2) Сервер генерирует токен: `header.payload.signature` (подпись через HMAC), 3) Клиент отправляет токен в заголовке `Authorization: Bearer <token>`, 4) Сервер верифицирует подпись без обращения к БД.

Преимущества токенов перед сессиями: 1) **Stateless** — сервер не хранит состояние сессии (масштабируемость), 2) **Горизонтальное масштабирование** — любой сервер может верифицировать токен, 3) **Мобильность** — токен работает across доменов и устройств, 4) **Безопасность** — подпись защищает от подделки, срок действия ограничивает ущерб при утечке.

Для масштабируемых систем: сессионный подход требует синхронизации сессий между серверами (Redis, sticky sessions), JWT работает без дополнительной инфраструктуры.

5. Заключение

По итогам выполнения лабораторной работы были решены все поставленные задачи. Спроектирована и реализована реляционная база данных для хранения юридических документов с ORM-интерфейсом. Реализован полный NLP-конвейер, включающий предобработку, извлечение именованных сущностей и автоматическую классификацию. Разработана поисковая подсистема типа поиск по метаданным и REST API с тремя эндпойнтами. Выполнено специализированное задание: t-SNE визуализация.

Собственные выводы студента

Наиболее успешно реализован: REST API на FastAPI с Pydantic-валидацией и автоматической документацией Swagger. Интеграция с SQLAlchemy ORM обеспечивает бесшовную работу с БД.

Ограничения: синтетическая природа корпуса (реальные данные были бы более шумными), относительно малый объём выборки (300 документов), отсутствие семантического поиска.

Методы улучшения: 1) Увеличение корпуса (1000+ документов), 2) Предобученные эмбединги (RuBERT) для семантического поиска, 3) Ансамблирование моделей (Random Forest + Gradient Boosting), 4) Гибридный поиск (FTS + векторный).

Масштабирование: контейнеризация (Docker), горизонтальное масштабирование API, кэширование частых запросов (Redis), асинхронные запросы к БД (AsyncSession).

Список литературы

- [1] Материалы Лекций 1–7 по дисциплине «Информационно-поисковые системы». 2025.
- [2] Документация библиотеки SQLAlchemy. URL: <https://docs.sqlalchemy.org>
- [3] Документация фреймворка FastAPI. URL: <https://fastapi.tiangolo.com>
- [4] Документация библиотеки scikit-learn. URL: <https://scikit-learn.org/stable/documentation.html>
- [5] Документация библиотеки pymorphy3. URL: <https://pymorphy3.readthedocs.io>
- [6] Документация SQLite FTS5. URL: <https://www.sqlite.org/fts5.html>
- [7] Manning C., Raghavan P., Schütze H. Introduction to Information Retrieval. Cambridge University Press, 2008. URL: <https://nlp.stanford.edu/IR-book>

Дополнительные источники

Документация библиотеки *t-SNE*: <https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>